agent

1.什么是ai agent（智慧体）？

agent = 大模型 + 工具调用 + 自主决策

agent能做到下面四点：1.感知环境 2.自主推理 3.调用工具 4.迭代执行

核心能力： 感知（Perception） 推理（Reasoning） 决策（Decision Making) 执行（Action）

2.agent的核心（工具调用）

Function Calling 整个过程分为四步：

1.告诉模型有哪些工具可以使用

2.模型觉得是否需要调用工具

3.外部程序执行工具

4.把结果返回给模型，生成最终回答

3.Memory 记忆系统

短期记忆：上下文，以及对话内容的叠加

长期记忆：向量数据库 结构化数据库 文件系统

4.Workflow(工作流) & Agent（智能体）

Workfow： 执行路径由代码预先定义好，适合生成画家 可预测 可控 当时灵活性有限

Agent：LLM自主执行路径 具有黑盒效应 下一步是不可预测的，不过胜在灵活 可以一堆开放式和未知场景

5.Agent架构 最佳实践

ReAct（最主流）：先思考 在行动，然后观察效果，再思考，再推理...直到任务完成

Plan and Execute :先规划 在执行 适合复杂的多步骤任务

Reflection：自我反思与改进 在执行完成任务之后会回顾和评估自己的表现

6.Mcp（模型上下文协议） 工具调用的开放标准

三大能力：Tools （工具/函数） Resources （资源）Prompts（提示词模板）

相当于agent中的Usb接口 处处适配

7.A2A agent之间的通信标准

A2A(Agent to Agent Protocol)： 相当于商业协议 明确的规定了自己的职责范围

核心：Agent Card（名片） Task（任务） Artifact（产出物）

8.Skills 技能

skills相当于是一本菜谱，不能让agent得到直接的答案，却规定了agent怎么样来自己做菜

为什么会出现skills？ 为了解决agent原有的四大问题

1.重复提醒 2.知识不可复用 3.质量不稳定 4.Context 窗口压力

Skills的核心：渐进式加载 （1.发现 2.激活 3.执行）

实际的应用场景 ： 1，团队编码规范统一 2.自动化测试 3.安全审计 4.文档生成

目前在实际的企业agent应用当中，已经出现了完整的三层架构（MCP+Skills+A2A）

A2A：agent之间如何发现 通信 委派任务

Skills：知识与流程

MCP：工具与数据连接

9.多Agent协作

在Java当中，当任务过大时，我们就可以由单线程转变成为多线程采用分片策略来提高系统对大任务的处理能力，agent也是如此。

Agent协助模式：1.主从模式 2.对话模式 3.流水线模式

在Java当中，多线程会带来很多的挑战，如：

- 线程安全问题，数据错乱
- 死锁，程序卡死
- 上下文切换开销大，效率降低
- 活锁、线程饥饿
- Bug 随机难复现，调试困难

在多agent当中也是如此：1.通信成本 2.协调难度 3.状态同步 4.错误传播 5.调试困难

所以在使用多anent开发的时候我们需要考虑，有必要使用吗？

10.Agent开发框架

Java：LangChain4j Sping ai LangGraph4j

python：LangChain LangGraph

11.Agent的应用场景

agent技术正在渗透各行各业，以下是一些典型的应用场景

1.智能客服 2.代码助手 3.数据分析 4.自动化办公 5.软件开发

LangChain4j

主要是代码实践 ，就不在这里赘述。有时间会整理代码上传github

Spring ai

同上